Random Machine Learning Notes

Three-Page Quick Study Report

This short report covers three useful machine learning topics: feature scaling, decision trees, and model evaluation. The goal is conceptual understanding rather than exhaustive mathematical detail.

1. Feature Scaling

Feature scaling changes the numerical range of input features so that variables with large numeric values do not dominate variables with smaller values. It is especially important for algorithms that depend on distances or gradient-based optimization.

Common Scaling Methods

• **Standardization:** transforms a feature so that it has approximately zero mean and unit standard deviation.

• **Min-Max Scaling:** maps values into a selected range, commonly 0 to 1.

• **Robust Scaling:** uses statistics such as the median and interquartile range and can be useful when outliers are present.

Algorithms that often benefit from scaling

• K-Nearest Neighbors (KNN)

• K-Means clustering

• Support Vector Machines

• Neural networks

• Gradient-based optimization methods

Tree-based models generally do not require feature scaling because their decisions are based on threshold splits rather than distances between feature values.

2. Data Leakage

Data leakage occurs when information that should not be available to a model during training becomes available through the training process. Leakage can produce unrealistically strong validation or test performance.

A common example is fitting a preprocessing transformation on the entire dataset before splitting into training and test sets. Information from the test set can then influence the transformation used during training.

3. Decision Trees

A decision tree makes predictions by repeatedly splitting data according to feature-based rules. Each internal node represents a decision, branches represent outcomes of the decision, and leaf nodes produce predictions.

How a Tree Learns

• Start with the complete training dataset.

• Evaluate possible feature-based splits.

• Select a split that improves the chosen impurity or loss criterion.

• Repeat the process on the resulting subsets.

• Stop according to conditions such as maximum depth, minimum samples, or insufficient improvement.

Classification Trees

For classification, common splitting criteria include Gini impurity and entropy. The tree attempts to create child nodes that are more homogeneous with respect to the target classes.

Regression Trees

For regression, a tree divides the feature space into regions and predicts a numerical value for each final region. A common objective is to reduce squared prediction error within the resulting groups.

Overfitting in Decision Trees

A very deep tree can memorize noise in the training data. Limiting tree depth, requiring a minimum number of samples in nodes, or using ensemble methods can reduce overfitting.

4. Random Forest

Random Forest combines many decision trees and aggregates their predictions. Each tree is trained using a randomized sample of the training data and a randomized subset of features at split points.

The main idea is that many diverse trees can produce a more stable model than one highly flexible tree. For classification, predictions are commonly combined through voting; for regression, predictions are commonly averaged.

5. Model Evaluation

Model evaluation measures how well a machine learning model performs for its intended task. The correct metric depends on whether the task is classification, regression, ranking, or another problem.

Classification Metrics

• **Accuracy:** proportion of predictions that are correct.

• **Precision:** among predicted positives, the proportion that are actually positive.

• **Recall:** among actual positives, the proportion correctly identified.

• **F1-score:** harmonic mean of precision and recall.

Regression Metrics

• **MAE:** average absolute prediction error.

• **MSE:** average squared prediction error.

• **RMSE:** square root of MSE, expressed in the target's original units.

• **R^2:** describes the proportion of target variation explained relative to a baseline.

6. Cross-Validation

Cross-validation provides a more reliable estimate of model performance by repeatedly dividing the available training data into training and validation portions.

In k-fold cross-validation, the data are divided into k folds. The model is trained on $k-1$ folds and validated on the remaining fold. This process is repeated so each fold serves as validation data, and the results are aggregated.

Why use it?

• It uses available training data efficiently.

• It provides a less dependent estimate than a single train-validation split.

• It is useful for comparing models and tuning hyperparameters.

7. Practical ML Reminder

A good machine learning workflow is not simply about choosing a powerful algorithm. Data quality, preprocessing, leakage prevention, suitable evaluation metrics, validation strategy, and understanding the problem are equally important.

Core workflow: Understand the problem → Prepare data → Split data → Preprocess correctly → Train → Validate → Tune → Test → Interpret.